FoodStore - Segunda entrega parcial del TPI

Materia: Base de Datos II

Integrantes: Franco Sampieri, Mauro Di Pietro, Tomas Martinez y Renzo Victoria

Motores: PostgreSQL 17, MongoDB y SQLite

Repositorio: https://github.com/francosampieri/TP2_BDD2_FoodStore

Los resultados citados se tomaron de pruebas ya documentadas en cada unidad. La programación diaria del respaldo quedó preparada como script, pero no se instaló como tarea de Windows ni se observó una ejecución automática.

1. Qué implementamos y cómo lo probamos

4 - FNBC y desnormalización

Trabajo: Analizamos ControlLoteAlmacen, sus dependencias funcionales y la violación de FNBC. Separamos la relación en R1 (responsable y depósito) y R2 (lote y responsable). También creamos una vista materializada para el top de categorías vendidas en el día.

Prueba y resultado: La reunión reconstruyó las tres filas originales y ambos EXCEPT devolvieron 0 filas. En una copia nueva, ROLLBACK quitó todos los objetos creados por la migración; al repetirla, el COMMIT conservó 3 filas originales y 3 reconstruidas, con EXCEPT post-commit en 0/0. Para dos detalles del día, la consulta directa tardó 4,335 ms y la lectura de la vista 1,561 ms.

5 - Administración y seguridad

Trabajo: Definimos roles de lectura, escritura, soporte y reportes, con permisos por columna para proteger usuario.contrasena. Configuramos auditoría y probamos funciones de autenticación con un usuario sintético.

Prueba y resultado: Nueve comprobaciones de privilegios dieron lo esperado. Una prueba de pedido con app_web bajó stock de 17 a 16 y el ROLLBACK lo devolvió a 17. El simulacro registró tres intentos fallidos, uno exitoso y un GRANT admin_datos TO app_web denegado; tras revertirlo quedaron 0 usuarios sintéticos y 0 membresías nuevas.

6 - Mantenimiento y recuperación

Trabajo: Creamos scripts de respaldo custom, chequeo de salud, mantenimiento y restauración de prueba. Ejecutamos un simulacro de recuperación en una base separada.

Prueba y resultado: pg_dump y pg_restore --list terminaron sin error. foodstore_restore_test conservó los conteos 8 categorías, 9 usuarios, 25 productos, 21 pedidos y 37 detalles. El RTO observado fue **31,96 s**; el respaldo tenía **22 min 37 s** de antigüedad estimada al iniciar (no es pérdida real medida). Un ANALYZE manual de pedido actualizó sus estadísticas sin usar VACUUM FULL.

7 - MongoDB

Trabajo: Modelamos pedidos con líneas embebidas y migramos una muestra de cinco pedidos y ocho detalles a MongoDB Compass. Preparamos consultas equivalentes en PostgreSQL.

Prueba y resultado: Las consultas por estado TERMINADO y producto 9 devolvieron los mismos IDs en ambos motores: 2, 3 y 12 para estado; 3 y 12 para producto. La muestra permite comparar funcionalidad, no rendimiento.

8 - SQLite e integración

Trabajo: Creamos una base local STRICT con catálogo, borradores, ítems y cola de sincronización. Importamos dos CSV desde la vista PostgreSQL y simulamos conflictos sin crear ventas.

Prueba y resultado: C2 detectó cambio de precio y se resolvió con un nuevo UUID; C3 rechazó falta de stock; C4 rechazó una baja lógica; C5 quedó aprobado de forma simulada; C6 rechazó UUID duplicado; C7 confirmó CHECK, FK y reversión. PostgreSQL conservó 21 pedidos y stock total 785.

2. Decisiones de diseño

La descomposición de la Unidad 4 elimina la repetición de DepositoID causada por la dependencia ResponsableControlID -> DepositoID. La unión R1/R2 es sin pérdida. La dependencia (LoteID, DepositoID) -> ResponsableControlID no queda garantizada automáticamente por las tablas separadas; el informe de la unidad deja explícito ese límite. Kiro especificó la migración reversible y OpenCode la plasmó en sql/tpi_u4_migracion_reversible.sql. En foodstore_tp_u4_mig, la primera ejecución terminó con ROLLBACK y to_regclass confirmó que la tabla original y R1 no existían. La segunda terminó con

COMMIT; los dos EXCEPT posteriores devolvieron 0 filas. El plan de reversión después del commit retira vista, R2 y R1, y conserva la tabla original y sus maestras.

La vista materializada es una desnormalización controlada: el resumen puede obtenerse de las tablas base, pero se almacena para evitar repetir joins y agregación en cada lectura. Se creó un índice único para admitir REFRESH ... CONCURRENTLY y se comprobó equivalencia. CURRENT_DATE se evalúa al refrescar; por eso el panel puede mostrar datos atrasados entre refrescos y la vista no sirve para confirmar ventas ni stock.

En seguridad aplicamos mínimo privilegio: app_web no lee contraseñas ni borra pedidos; admin_datos lee reportes y no ejecuta sp_crear_pedido. El procedimiento y los triggers son SECURITY INVOKER, por lo que el rol de escritura también necesita permisos directos para el flujo transaccional. No afirmamos que solo pueda escribir mediante el procedimiento. El análisis asistido por IA del simulacro se contrastó con el log y la matriz de permisos: se rechazó la inferencia de que leer dos columnas de usuario implicaba acceso a toda la tabla.

PostgreSQL sigue siendo la fuente de verdad de stock y pedidos. MongoDB se usó para una lectura documental de pedidos completos; un pedido embebido no hace atómica, por sí solo, una actualización de stock en otro documento. SQLite conserva el catálogo visto y borradores durante la desconexión. SIMULADO_APROBADO solo expresa el resultado de la comparación local: la venta real requeriría volver a validar stock y precio en PostgreSQL, además de deduplicar el UUID en una transacción del servidor.

3. Mantenimiento y recuperación

El respaldo custom de foodstore se comprobó con pg_restore --list y se restauró en foodstore_restore_test, sin reemplazar la base principal. El script de restauración midió desde antes de crear la copia hasta completar los controles; por eso los **31,96 s** son un RTO del simulacro. La antigüedad del respaldo al inicio fue una estimación de RPO, no una medición de datos perdidos. La primera versión del script falló al interpretar una salida vacía de PowerShell 5.1; se corrigió y el simulacro se repitió con éxito.

El plan propone respaldo y chequeo de salud diarios, más revisión de mantenimiento y restauración de prueba semanales. scripts/programar_backup_foodstore.ps1 deja preparada la tarea diaria, pero el equipo decidió no registrarla en esta computadora. Por tanto, quedó probado el respaldo manual y la restauración; la ejecución automática permanece sin verificar.

4. Uso de IA y revisión final

Kiro se usó para especificar el análisis FNBC, el esquema de roles y los requisitos EARS de la tablet. OpenCode generó scripts y documentación inicial de las unidades 4, 5, 6 y 8, y revisó por lectura estática el conjunto de scripts de las unidades 1 a 8. Codex se usó adicionalmente para contrastar planes, CSV, consultas y resultados reales. Las decisiones por herramienta, prompt o spec se registran en docs/tpi/duia_tpi_segunda_entrega.md y en las bitácoras de cada unidad.

La revisión final aceptó y corrigió el precio unitario arbitrario en un INSERT de detalle: antes aceptaba 0.01 y después tomó el precio real 2500.00 y subtotal 5000.00. También corrigió un borrador vacío que quedaba pendiente en SQLite; ahora se rechaza. Se descartó la afirmación de que un índice parcial de TP5 era inútil: el plan real sí lo usó y la consulta pasó de 31,581 ms a 2,767 ms. La decisión humana sobre todos los hallazgos está en docs/tp8/informe_tp8.md.

5. Alcance de la entrega

Los scripts, las pruebas y la documentación de las unidades 4 a 8 están reunidos en el repositorio. El único límite conocido frente a la consigna es el respaldo programado: existe el script para registrarlo, pero no hay evidencia de una tarea instalada ni de una ejecución automática. La defensa puede apoyarse en las pruebas manuales y los informes de cada unidad, sin presentar la automatización como verificada.

Referencias principales: docs/tp_u4/informe_tp_u4.md, BITACORA.md, docs/informe_auditoria.md, docs/informe_incidente.md, informe_u6.pdf, docs/tp7/informe_tp7.md, docs/tp8/informe_tp8.md.